Homework 5

William Flanders*
Haverford College
(Dated: March 1, 2023)

1. CALCULATING BESSEL FUNCTIONS WITH RECURSION

We want to investigate the method of calculating nth order Bessel functions using recursion. We want to compare this method with calculating the functions using Simpson's Rule. Building upon what we've already done using integration, the idea is that we want to create a program that can find these functions recursively, create some plots of the two methodologies against one another, and analyze the fractional error between the two.

First, lets detail the recursive method of finding an nth order Bessel function. The technique can be defined mathematically as

$$J_{n+1} = \frac{2n}{x} J_n - J_{n-1} \tag{1}$$

where n is the order. This type of recursion is defined as upwards since the next value is dependent on the current and/or previous values. To create a program that moves recursively upwards, we will need J_0 and J_1 as base cases. We will have to use the previously analyzed integral method (Simpson's Rule) to find these base cases as $n > 0$. Furthermore, the equation is undefined when $x = 0$. This recursion technique, as we'll soon see when we look at our error analysis, holds only when $x > 0$ and grows remarkably more accurate as $x \to \infty$.

We will look to compare the $n = 2, 3, 4$th order Bessel functions from both methods. We will look to compare each function for a range of x values (from 10 to about 70).

Let's begin by writing pseudo code:

```
import dependencies

def Bessel method using Simpson's:
    return J_n

def Bessel method using recursion:
    base cases of J_0 and J_1
    recurssive case:
        loop through values of n
            use Equation 1 for J_n+1
            redefine J_0 and J_1
return J_n+1

error = absolute value(Simpson's[ith element]
                                recursion[ith element])

plot(Simpson's vs. recursion)
plot(error)
```

The resulting figures are shown below:

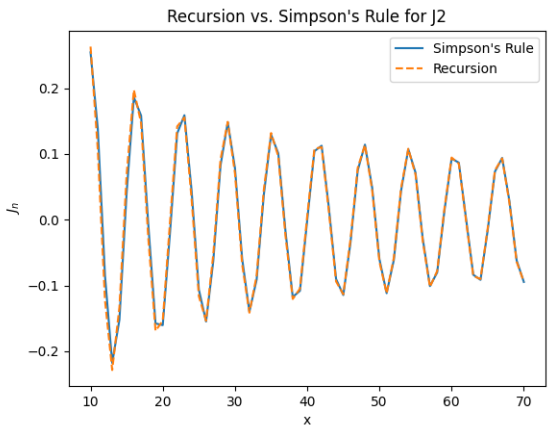

FIG. 1: 2nd Order Bessel Function

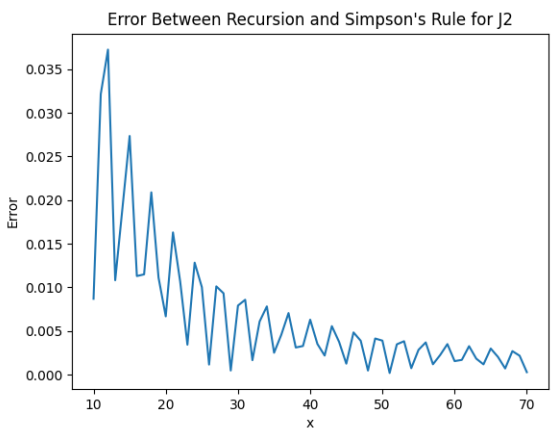

FIG. 2: Error between Simpson's Rule and Recursion for J_2

For all $n = 2, 3, 4$ orders of Bessel function, the figures reveal some clear behaviors. First, in general both methods share similar outputs for the range of x values from 10 to 70. We can see this in Figures 1, 3, and 5. The two plots are nearly on top of one another. Furthermore, as we can see from the error plots, Figures 2, 4, and 6, the deviations shrink nearly exponentially as x continues

*Electronic address: wflanders@haverford.edu; URL:

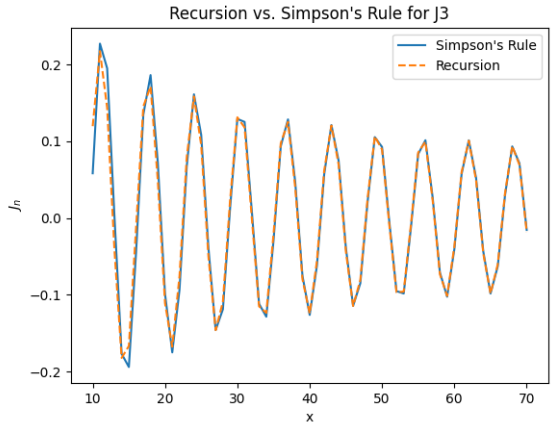

FIG. 3: 3rd Order Bessel Function

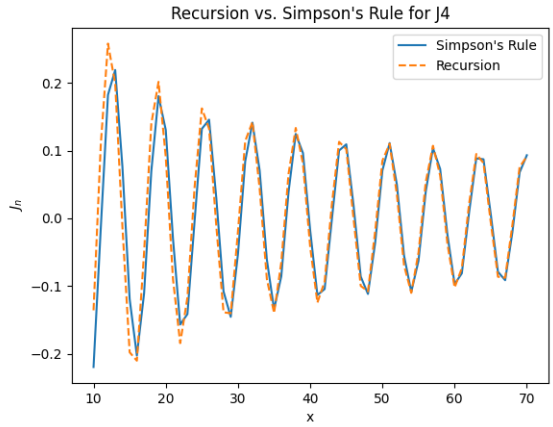

FIG. 5: 4th Order Bessel Function

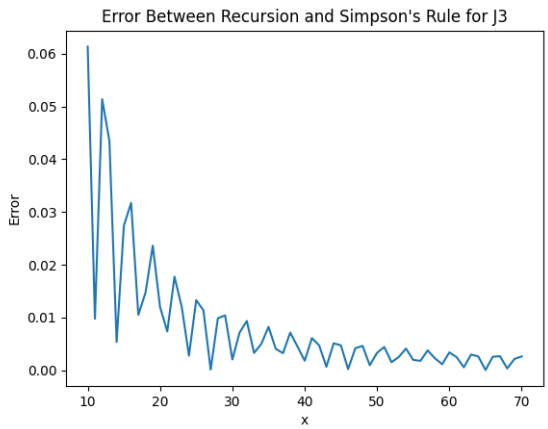

FIG. 4: Error between Simpson's Rule and Recursion for J_3

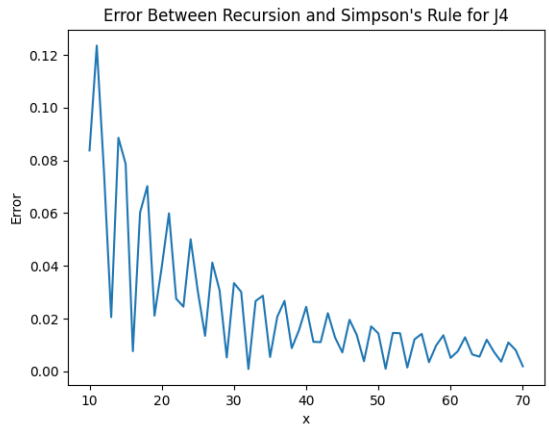

FIG. 6: Error between Simpson's Rule and Recursion for J_4

to grow. This implies that recursion, as x grows larger, begins to converge to the same output as Simpson's rule. Therefore, an optimal method, for when x is very large, can swing either recursion or Simpson's rule depending on computational costs. However, for when x is very small, Simpson's rule exhibits far less variation and is thus more reliable.

2. NEWMAN EXERCISE 5.13

We want to investigate the behavior of the wave function of the nth energy level of a quantum harmonic oscillator in one dimension. A quantum harmonic oscillator can be summarized as a spinless point particle that sits in a quadratic potential well i.e. its energy curve can be comparable to a classical harmonic oscillator (and therefore modeled as such). The equation of the wave function can be given by

$$\Psi_n(x) = \frac{1}{\sqrt{2^n n! \sqrt{\pi}}} e^{-x^2/2} H_n(x) \tag{2}$$

where $H_n(x)$ is the nth Hermite polynomial. We can describe Hermite polynomials recursively where

$$H_{n+1} = 2xH_n(x) - 2nH_{n-1} \tag{3}$$

and H_{n+1} is the n+1th Hermite polynomial. We will want to create a program that is capable of calculating

the nth Hermite polynomial, use it in a wave function method, and then plot a series of descriptive figures. Furthermore, we will want to calculate the root mean squared position of the wave function. This gives us the quantum uncertainty of a particle's position in the nth energy level. We will do this using Gaussian quadrature. We start with pseudo code:

```
import dependencies

def H(n,x):
    base cases for n=0 and n=1
    recursive case for when n > 1:
        loop through values of n:
            calculate the n+1th Hermite polynomi

def Psi(n, x):
    formula for Psi using H(n, x)

def rsmp():
    x_squared = Gaussian quadrature to calculate
    return sqrt(x_squared)

plot()
```

We will be looking to create a plot for the wave function of $n = 0, 1, 2, 3$ energy levels and from $-4 \leq x \leq 4$. We will also want to create a plot of the wave function when $n = 30$ and when $-10 \leq x \leq 10$. Finally, we'll calculate for the root squared mean position when $n = 5$ and using $N = 100$ steps for our Gaussian quadrature. We will use the quad() function from the library scipy to perform the integral. Figures 7 and 8 showcase our plots.

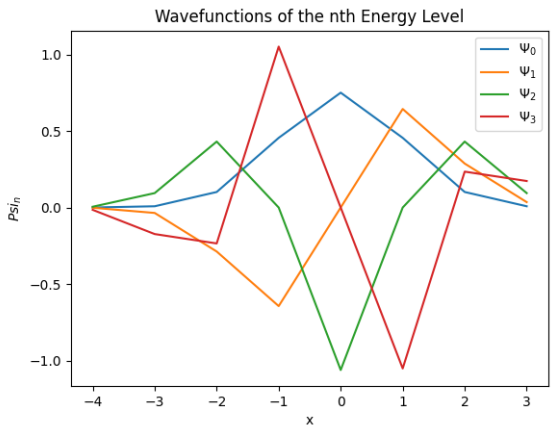

FIG. 7: The Wave Function for $n = 0, 1, 2, 3$ Energy Levels

We can see in Figure 7, as the energy level increases, the wave function moves from a clear sinusoid to being a composition of waves (where Ψ_0 looks like a perfect sin wave). When $n = 30$, the wave function takes a distinct shape where at $x = 0$, the function plummets to nearly -600000, symmetrically rises to roughly 2000000, before

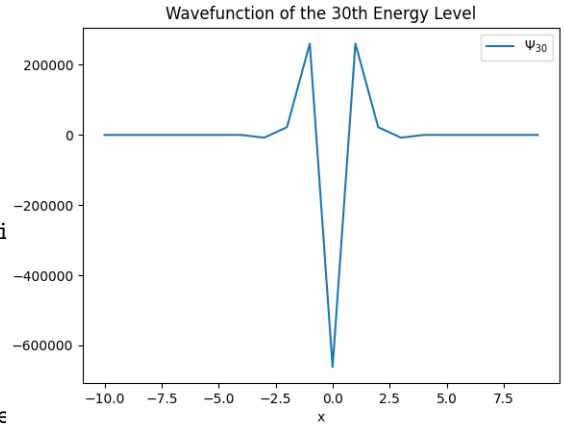

FIG. 8: The Wave Function for $n = 30$ Energy Level

settling to 0 again. Finally, for our quantum uncertainty in the particle's position, we returned a value of 3.041.... This is somewhat off from the theoretical prediction of 2.3 with a deviation of 0.741.

3. SURVEY

The problem set took roughly 5-6 hours. Both exercises were the right amount of challenging but also rewarding to complete. I felt I was able to really hone my ability to move upwards recursively for the $n+1$th term. More practice plotting is always welcome as well.